The bridge: what it is, and what building it would mean

REJECTED BY THE OWNER — 13 November 2026. There is no Discord bridge in this product and there will not be one. This document is kept as the record of what the idea was, what it would have cost, and why it was declined, so it is not re-proposed in six weeks with a fresh coat of paint. Nothing below is planned work.

The short answer first, because the idea is easier to judge once it is concrete.

A bridge is a small program that sits between two chat systems and copies messages from one to the other, in one direction or both. Ours would connect one channel in Nightgrove to one channel in a Discord server, so a message typed in either place appears in both, carrying the name of the person who actually wrote it.

Where	What happens
Nightgrove, `#the-porch`, a server you own	someone types "evening all"
the bridge, one small process	reads it, and posts a copy on the other side
Discord, `#wizardry-bridge`, someone else's server	the copy appears, carrying the name of the person who typed it, and can be answered there

Nothing is shared behind the scenes. The bridge reads a message in one place and posts a copy in the other, with the original author's name attached. Two consequences follow from that, and they are the whole of the design: **the copy is a copy** (an edit in one place does not silently change the other unless we build that), and **the message has left the building** the moment it crosses.

1. What ours would do

Does	Does not
Relay text both ways between one Nightgrove channel and one Discord channel	Relay every channel, or sync the guild's structure
Keep the sender's display name and avatar on the copy	Pretend the copy was written in the place it landed
Carry attachments as links or re-uploads, per a setting	Mirror attachments by default (storage and copyright)
Mark each copy as bridged, visibly	Hide that a message came from outside
Queue when either side is down, and say so	Drop messages silently
Forget ids after a retention window	Keep a permanent map of who said what where

One channel. Named. Both ways. That is the whole scope — and the reason it is worth planning at all is that the people you would invite are already in a Discord, so the instance does not have to win an argument to be useful on its first day.

2. Three ways to build it, and what each costs

Shape	What it needs	Good	Bad
Outbound only, via a Discord webhook	One Discord webhook URL; a Nightgrove webhook or bot to post from	An afternoon. No bot account, no gateway, no token to rotate	One-way. Discord cannot answer
Two-way, with a Discord bot	A Discord application and bot token from the Developer Portal (**your hands**), the message-content intent, a Nightgrove bot account	Real conversation across the boundary, names preserved	A week of work, two tokens to keep, Discord's rate limits to respect
A hosted relay (Matterbridge-shaped)	A third service to run and patch	Existing tooling	Another daemon to own; its mapping rules are not ours

The recommendation is the first, then the second: **ship outbound in an afternoon and discover whether anyone replies**, before spending a week on the half that needs a bot token.

3. The hard parts, each with its rule

- **Identity.** A copy must carry the original author's name, and must not create an account in Nightgrove for every Discord user — so a bridged message is *posted by the bridge* and *labelled with the author*. On the Discord side a webhook overrides the name and avatar, which is why the outbound half is cheap.
- **Loops.** The bridge must never re-post its own output. Every copy carries the id of the message it came from, and the bridge ignores anything carrying a bridged marker. This is the failure that melts a bridge in ten seconds if it is got wrong.
- **Edits and deletes.** In the MVP the copy is a copy: an edit does not chase. The bridge says so in its own docs rather than silently diverging. Deletes are the exception — a delete removes the copy, because leaving a deleted message readable outside is a privacy failure, not a staleness bug.
- **Attachments.** Links by default; a size ceiling; never a silent failure. A message whose attachment could not be mirrored still arrives with a note.
- **Rate limits.** Discord enforces per-channel limits and will punish a hammer; the bridge queues and drains, and a backlog is visible on the Nightgrove side rather than lost.
- **Backfill.** Turning a bridge on does not replay history. A channel starts empty on the far side and says so.

4. Failure modes, and what a person sees

When	What happens	What the person sees
Discord is down	The bridge queues; Nightgrove keeps working	Copies arrive late, with their original times
The token is revoked	The bridge stops and refuses to retry blindly	A one-line notice in the bridged channel

When	What happens	What the person sees
A loop is detected	The bridge halts that channel rather than spinning	A notice, and nothing is re-posted
A copy fails validation	That message is skipped and logged	A gap, with a note in the channel's log
The bridge process dies	Nothing is copied until it returns	Messages resume; the gap is reported

The rule underneath all of them: **a bridge that fails quietly is worse than no bridge**, because people keep talking into a void.

5. The MVP, with pass criteria

Step one — outbound, in an afternoon, no bot token. A Nightgrove webhook posts into a Discord webhook; every message in one channel crosses.

- Pass: a message typed in Nightgrove appears in Discord within five seconds, with the right name, and a second message does not appear twice.
- Fails if: the copy carries the bridge's name instead of the author's, or a restart duplicates anything already sent.

Step two — inbound, about a week. A Discord bot listens on one channel and posts into Nightgrove through a bot account.

- Pass: a message typed in Discord appears in Nightgrove within five seconds, labelled as bridged, and a reply typed in Nightgrove reaches Discord without a loop.
- Fails if: any message round-trips twice, or a Discord user's name is used to impersonate them into a Nightgrove conversation they are not part of.

6. What it costs to run

A process, two secrets, and a table of message ids with a retention window. The process is small enough to live on the existing machine; the secrets are the part that needs care, and the token is yours to create and revoke.

7. The real objection: the message leaves

Everything else in this plan keeps conversation inside a server you own. A bridge puts a copy of your conversation on someone else's infrastructure, under their terms, and a Discord user reading the bridged channel sees the Nightgrove side without ever joining.

That is not a technical problem to solve; it is a decision to make. What the plan should carry, if it is built:

- The channel is named as bridged, and every copy says where it came from.
- Nothing is bridged by default; a channel is chosen, by you, one at a time.
- Anything said in the bridged channel should be treated as public to both rooms. Said plainly, in the channel's topic, rather than buried in a document.

- Turning the bridge off leaves no copy behind on the far side — or leaves the whole history, one of the two, and never a half-deleted thread.

8. The gate

The idea jumps the queue only if **a named person, in a named Discord channel, would use the bridged room.** Without that name it is infrastructure for an audience that does not exist, which is the exact mistake `IDEAS.md` was written to avoid.

So the honest sequence is:

1. Name the person and the channel.
2. Build step one — an afternoon, outbound only.
3. Watch for a week. If nobody replies in Discord, stop: the bridge answered a question nobody asked, and the week step two would have taken belongs to the queue.
4. If they do reply, step two is justified, and the queue gains its first real contender.

9. Decisions that are yours

1. **The person and the channel** — the gate above, and the only one that matters first.
2. **One channel or two**: a dedicated bridged room, or the bridge woven into an existing one.
3. **Attachments**: links, re-uploads, or refused.